

Algebraic Cipher Types: Confidentiality Trade-offs in Type Constructors over Trapdoor Computing

Alexander Towell
lex@metafunctor.com

March 2026

Abstract

Cipher types form an algebra: starting from primitive ciphered values, the type constructors of ordinary programming (products, sums, exponentials) build complex cipher types from simpler ones, with each constructor inducing a specific confidentiality cost absent in plaintext programming. We characterize the algebra constructor-by-constructor: products leak correlations, sums force a categorical impossibility (tag hiding and untrusted pattern matching cannot coexist), and the cipher exponential is the cipher map abstraction itself, closing the algebra back on the trapdoor framework. A uniform information-theoretic bound holds across the algebra: the conditional entropy of the latent value given the untrusted machine’s view is at least $H(X) - \log_2 |\text{orbit}|$, where the orbit is the set of cipher values reachable under the operations the untrusted machine holds. A typed-composition discipline turns this bound into a design-time budget: the type of a cipher program determines how deeply its orbit can grow. We realize cipher programs as Python expression-tree decompositions with cipher-map cut points (a `@cipher_node` decorator marks the cipher subtrees; unmarked nodes run as plain code over opaque bit strings) and evaluate the cipher Boolean specialization on a 20 Newsgroups Boolean search task, where the cut-point discipline measurably bounds confidentiality loss.

1 Introduction

Trapdoor computing is a paradigm for evaluating functions on data whose meaning is hidden behind a one-way trapdoor (a cryptographic hash). A trusted machine encodes plaintext values into cipher values—opaque bit strings—and hands them, together with cipher maps (total functions on bit strings), to an untrusted machine. The untrusted machine evaluates the cipher maps and returns results. Without the trapdoor (the secret seed), it cannot decode the results or determine what function it computed. The formal framework of cipher maps, their four defining properties (totality, representation uniformity, correctness, composability), and three concrete constructions (HashSet, entropy map, trapdoor boolean algebra) are developed in [18].

Ordinary programming languages build complex types from simpler ones using a small set of type constructors: products ($A \times B$, pairs and records), sums ($A + B$, tagged unions and variants), and exponentials ($A \rightarrow B$, functions). In plaintext computing, these constructors are transparent: a product exposes both components, a sum carries a tag that enables pattern matching, and a function is called by providing an argument. When values are hidden behind a trapdoor, each of these operations introduces a confidentiality cost that has no plaintext analogue.

This paper identifies and formalizes those costs. The main contributions are:

1. **An algebra of cipher types** (Section 4). We survey the standard type constructors over cipher values (void, unit, product, sum, exponential) and identify the confidentiality cost

each one introduces. Products leak correlations under component-wise encoding (Proposition 4.1); sums force an impossibility between tag hiding and untrusted pattern matching (Theorem 4.2); the cipher exponential is exactly the cipher map abstraction (Proposition 4.3), closing the algebra back on the trapdoor framework, with its own granularity trade-off (Proposition 4.4). The encoding-granularity knob (joint vs. component-wise) runs uniformly through the algebra; the costs differ by constructor.

2. **A uniform information-theoretic bound** (Section 5). We define the orbit closure of a cipher value under the operations available to the untrusted machine (Definition 5.1), prove monotonicity (Theorem 5.1), and derive an entropy-form confidentiality bound $H(X \mid \mathcal{V}_F(c)) \geq H(X) - \log_2 |\text{orbit}_F(c)|$ (Theorem 5.3) that is independent of the underlying cipher-map construction. A typed-composition discipline (Section 5.5) converts the bound into a design-time budget: the type of a cipher program bounds its orbit depth, and hence its worst-case confidentiality loss.
3. **Expression-tree realization** (Section 6). We realize the algebra at the program level: a cipher program is a composition of cipher maps and plain operations over opaque bit strings. Cipher-node annotations on a program’s expression tree mark which subtrees become cipher maps; unmarked nodes run as plain code on the cipher bytes. The placement of annotations selects the cut points, and the type of the cut-point cipher spaces determines the orbit budget per Proposition 5.4.

We deliberately avoid game-based or simulation-based cryptographic definitions. Privacy in trapdoor computing comes from the one-way hash and the representation uniformity of cipher values, not from access-pattern indistinguishability. This is not ORAM [8], FHE [7], or garbled circuits [21]; it is a distinct paradigm where totality and noise closure provide the confidentiality guarantees.

2 Related Work

Information flow type systems. The sum-type impossibility (Theorem 4.2) is closely related to the implicit flow problem in language-based information security [13]: a conditional branch on a secret value leaks information through the control flow path taken. Our result formalizes this for cipher values: pattern matching on a cipher sum type is an implicit flow that leaks the tag. The difference is that information flow type systems prevent the leak at compile time, while cipher types quantify it at the representation level.

Functional encryption. Functional encryption [3] allows computing $f(x)$ on an encrypted x by issuing function-specific decryption keys. Cipher maps differ in two ways: (1) they are information-theoretic (parameterized by $\eta, \varepsilon, \delta$) rather than game-based, and (2) they are approximate (the untrusted machine evaluates a total function with bounded error, not an exact decryption). The type-constructor trade-offs we identify (tag leakage for sums, correlation leakage for products) apply equally to functional encryption schemes that support algebraic operations.

Searchable symmetric encryption. SSE originates with Song, Wagner, and Perrig [16] and is placed on rigorous foundations by Curtmola et al. [5]. The Boolean search construction in Section 7 provides an alternative with information-theoretic parameters: cipher sets with cipher Boolean AND/OR/NOT, rather than encrypted indices with leakage profiles. Subsequent work established

that leakage profiles in SSE are attackable in practice: Islam, Kuzu, and Kantarcioglu [10] demonstrated access-pattern disclosure attacks; Cash et al. [4] and Grubbs, Ristenpart, and Shmatikov [9] mounted leakage-abuse attacks against deployed SSE and encrypted databases; Naveed, Kamara, and Wright [12] did the same for property-preserving encryption. Cipher maps avoid property preservation entirely and limit the adversary to the orbit-closure bound of Theorem 5.3.

Quantitative information flow. The entropy-form confidentiality bound of Theorem 5.3 places this work in the quantitative information flow tradition [15], where leakage is measured as conditional entropy or min-entropy of the secret given the adversary’s observations. Our contribution is to specialize this framework to cipher values under an algebraic operation set, yielding a construction-independent bound in terms of orbit size.

Related concepts. Cipher maps realize the oblivious evaluation of a computable function (after Turing [20]) through a total function on bit strings, and the cipher Boolean construction (Section 7) is structurally related to the Bloom filter [2], which it generalizes by allocating an explicit False region alongside the True region. Our experiments sit on the 20 Newsgroups corpus of Lang [11]. Maximizing-confidentiality analyses complementary to this paper’s information-theoretic bound appear in a companion work [19].

3 Preliminaries

We recall the cipher map abstraction from [18]. All notation follows that paper; we cite rather than re-derive.

Definition 3.1 (Cipher map [18, Def. 3.1]). A *cipher map* for a latent function $f : X \rightarrow Y$ is a tuple $(\hat{f}, \text{enc}, \text{dec}, s)$ where $\hat{f} : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a total function on n -bit strings, $\text{enc} : X \times \{0, \dots, K-1\} \rightarrow \{0, 1\}^n$ is an encoding function, $\text{dec} : \{0, 1\}^n \rightarrow Y \cup \{\perp\}$ is a decoding function, and s is a secret (the trapdoor). The untrusted machine holds $\hat{f}$. The trusted machine holds enc , dec , and s .

A cipher map satisfies four properties, parameterized by $(\eta, \varepsilon, \mu, \delta)$ [18, Sec. 4]:

1. **Totality.** $\hat{f}$ is defined on all 2^n inputs. Out-of-domain outputs are indistinguishable from uniform under the random oracle model [1].
2. **Representation uniformity** (δ -bounded). The marginal distribution of cipher values is δ -close (in total variation distance) to uniform over $\{0, 1\}^n$.
3. **Correctness** (η -bounded). For a random in-domain element x and random representation index k , $\Pr[\text{dec}(\hat{f}(\text{enc}(x, k))) \neq f(x)] \leq \eta$.
4. **Composability.** For cipher maps $\hat{f}$ (correctness η_f) and $\hat{g}$ (correctness η_g), the composition $\hat{g} \circ \hat{f}$ has correctness $\eta_{g \circ f} \leq 1 - (1 - \eta_f)(1 - \eta_g)$, with equality under the re-randomization condition of [18, Thm. 4.2].

The Bernoulli error model [17] provides the quantitative foundation: element-wise independence of errors reduces the composition analysis from exponentially many joint failure modes to per-element Bernoulli parameters. We use $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ throughout to denote a cryptographic hash modeled as a random oracle.

A cipher value $\text{enc}(v, k)$ is a cipher map for the constant function $f_v(x) = v$; we use “cipher value” and “cipher map from a trivial domain” interchangeably.

Definition 3.2 (Trusted and untrusted machines [18, Sec. 5]). The *trusted machine* T holds the trapdoor s , enc , dec , and can encode, decode, and inject filler queries. The *untrusted machine* U holds $\hat{f}$ and a collection of cipher values. U can evaluate $\hat{f}$ on any bit string; it cannot decode, distinguish real from filler queries, or determine the domain X .

Definition 3.3 (Cipher type). For a finite latent type X with encoding $\text{enc} : X \times \{0, \dots, K-1\} \rightarrow \{0, 1\}^n$, the *cipher type* of X is

$$\mathbf{C}(X) = \{\text{enc}(x, k) : x \in X, 0 \leq k < K(x)\} \subseteq \{0, 1\}^n.$$

We use $\mathbf{C}(X)$ for both the set and the type-theoretic constructor: $\mathbf{C}(X \times Y)$ denotes the cipher type for the latent product, and similarly for sums and exponentials. A cipher map $\hat{f} : \mathbf{C}(X) \rightarrow \mathbf{C}(Y)$ is the cipher form of the latent function $f : X \rightarrow Y$ (extended totally over $\{0, 1\}^n$ per Property 1).

4 Cipher Type Constructors

Cipher types form an algebra: starting from primitive ciphered values, the type constructors of ordinary programming (void, unit, product, sum, exponential) build complex cipher types from simpler ones. We examine each constructor in turn and identify its *confidentiality cost*: the irreducible information the untrusted machine learns when the constructor is realized over cipher values.

A common knob runs through the algebra. For every binary type constructor, the cipher type can be built by *joint encoding* (the constructed type as a single cipher value, $\mathbf{C}(A \star B)$) or *component-wise encoding* (the constructed type as a structure over the cipher types of its components, $\mathbf{C}(A) \star \mathbf{C}(B)$). Joint encoding hides more structure but demands a fresh cipher map for each operation that exposes it (projection, pattern matching); component-wise encoding makes operations available to the untrusted machine at the cost of leaking exactly the structural relationship that joint encoding was hiding. The cost is constructor-specific: products leak *correlations* (Proposition 4.1), sums leak the *tag* (Theorem 4.2), and exponentials leak the *computation pattern* (Proposition 4.4).

The same encoding-granularity knob extends to the orbit-closure analysis of Section 5: the operations a constructor exposes to the untrusted machine determine the size of the reachable cipher space, and the typed-composition discipline of Section 5.5 bounds the resulting confidentiality loss. Together, the per-constructor costs of this section and the uniform information-theoretic bound of the next combine into a confidentiality calculus over the cipher-type algebra.

4.1 Void and Unit

The void type $\mathbf{0}$ has no values. A cipher map $\hat{f}$ for a function with domain $\mathbf{0}$ is vacuously correct ($\eta = 0$) and carries no information—it is a total function on $\{0, 1\}^n$ that is never queried on a valid encoding. From the untrusted machine’s perspective, $\hat{f}$ is indistinguishable from a random function.

The unit type $\mathbf{1}$ has exactly one value, $\star$. A cipher value $\text{enc}(\star, k) \in \{0, 1\}^n$ encodes $\star$; with $K(\star) \geq 1$ representations, all cipher values are representations of the same underlying value. The untrusted machine knows that every valid cipher value encodes $\star$ but cannot distinguish valid encodings from noise (by totality). A cipher map $\hat{f} : \{0, 1\}^n \rightarrow \{0, 1\}^n$ for a function from $\mathbf{1}$ is a cipher constant: it maps every valid encoding of $\star$ to a cipher encoding of $f(\star)$.

4.2 Product Types

The product type $A \times B$ consists of pairs (a, b) with $a \in A$ and $b \in B$. In plaintext computing, a product comes with two projections $\pi_1 : A \times B \rightarrow A$ and $\pi_2 : A \times B \rightarrow B$. Under cipher encoding, we have two choices.

Joint encoding: $C(A \times B)$. Encode the pair (a, b) as a single cipher value $\text{enc}((a, b), k) \in \{0, 1\}^n$. The untrusted machine sees one opaque bit string. It cannot determine a or b individually, nor can it detect correlations between a and b . But the projections π_1, π_2 are not available: to extract a from a cipher encoding of (a, b) , one would need a cipher map $\hat{\pi}_1 : \{0, 1\}^n \rightarrow \{0, 1\}^n$ such that $\text{dec}_A(\hat{\pi}_1(\text{enc}((a, b), k))) = a$ —which requires a separate cipher map construction for each projection.

Component-wise encoding: $C(A) \times C(B)$. Encode a and b independently as $(\text{enc}_A(a, k_1), \text{enc}_B(b, k_2))$. The untrusted machine sees a pair of cipher values. Projections are trivially available (just take the first or second component). But the joint distribution of (a, b) is no longer hidden: even if each component has marginal representation uniformity ($\delta \approx 0$), the correlation between a and b is preserved in the joint distribution of the cipher pair. This is the encoding granularity principle of [18, Sec. 8.1]: coarser encoding hides more correlations but blocks more operations.

Proposition 4.1 (Product confidentiality trade-off). *Let A and B be finite types.*

1. *Under joint encoding $C(A \times B)$, the untrusted machine cannot distinguish (a_1, b_1) from (a_2, b_2) (up to δ), but projections require a cipher map (and thus a separate construction by the trusted machine).*
2. *Under component-wise encoding $C(A) \times C(B)$, projections are free, but an adversary observing N pairs $(\text{enc}_A(a_i, k_{1i}), \text{enc}_B(b_i, k_{2i}))$ can estimate the joint distribution of (A, B) to arbitrary precision as $N \rightarrow \infty$.*

Proof. Part (1): Under joint encoding with representation uniformity δ , each cipher value is δ -close to uniform over $\{0, 1\}^n$. Without the trapdoor, the untrusted machine cannot decode. Projections require mapping a cipher encoding of (a, b) to a cipher encoding of a ; this is a function from the code space of $A \times B$ to the code space of A , which is exactly a cipher map and requires construction by the trusted machine.

Part (2): Under component-wise encoding, the i -th observation is $(\text{enc}_A(a_i, k_{1i}), \text{enc}_B(b_i, k_{2i}))$. Even with perfect marginal uniformity ($\delta = 0$), each enc_A is a deterministic function of (a_i, k_{1i}) . If the same representation index is reused, repeated values produce identical cipher values; even without reuse, the number of distinct cipher values associated with each latent value is finite ($K(a)$ for each a). Given N observations, the adversary builds the empirical joint distribution over cipher-value pairs. Since each cipher value maps to at most one latent value (given a fixed representation index), the empirical joint distribution of cipher pairs converges to a mixture of the latent joint distribution. By standard convergence results, $N = O(|A| \cdot |B|/\xi^2)$ observations suffice to estimate the joint to accuracy ξ in total variation distance. $\square$

4.3 Sum Types

The sum type $A + B$ is a tagged union: each value is either $\text{inl}(a)$ for some $a \in A$, or $\text{inr}(b)$ for some $b \in B$. The tag (left vs. right) enables pattern matching: given a value of type $A + B$, one inspects the tag to determine which handler to apply.

In plaintext computing, the tag is always available and pattern matching is free. Under cipher encoding, this is no longer the case.

Theorem 4.2 (Sum-type impossibility). *Let A and B be non-empty finite types with $|A| \geq 1$ and $|B| \geq 1$. For cipher values of sum type $A + B$, the following two properties are mutually exclusive:*

1. **Tag hiding:** *the untrusted machine cannot determine, from a cipher value alone, whether it encodes an A -value or a B -value with advantage better than δ (the representation uniformity parameter).*
2. **Untrusted pattern matching:** *holding the A -handler $\hat{f}$ and the B -handler $\hat{g}$ separately, the untrusted machine can apply exactly one of them to a cipher value, choosing which as a function of that value, with no per-query assistance from the trusted machine.*

Proof. We prove the two directions by exhibiting the information-theoretic constraint.

Joint encoding ($C(A + B)$) achieves tag hiding but blocks pattern matching. Under joint encoding, each value $v \in A + B$ (whether $\text{inl}(a)$ or $\text{inr}(b)$) is encoded as a single cipher value $\text{enc}(v, k) \in \{0, 1\}^n$. Representation uniformity (δ -bounded) guarantees that the marginal distribution of $\text{enc}(v, k)$ over random v and k is δ -close to uniform on $\{0, 1\}^n$. In particular, the set of cipher values encoding A -values and the set encoding B -values are not efficiently distinguishable by the untrusted machine: both are subsets of $\{0, 1\}^n$, and their union is δ -close to covering $\{0, 1\}^n$ uniformly. Therefore the tag (which variant) is hidden up to δ .

Now suppose the untrusted machine could nonetheless perform dispatch: holding $\hat{f}$ and $\hat{g}$ separately, it applies exactly one of them to $c = \text{enc}(v, k)$, choosing which as a function of c . That choice is a selector $s : \{0, 1\}^n \rightarrow \{A, B\}$ with $s(\text{enc}(\text{inl}(a), k)) = A$ for all a, k and $s(\text{enc}(\text{inr}(b), k)) = B$ for all b, k . But such an s is exactly a distinguisher between A -encodings and B -encodings, which exists only if the two sets of cipher values are separable without the trapdoor. If s is correct with probability $1 - \gamma$, then the tag is leaked with advantage at least $1 - \gamma - \delta$, contradicting tag hiding when γ is small. Note the obligation is to form the *selector*, not merely to compute the eliminator's result: the result of the case analysis can be produced as a cipher value with the tag hidden (by a fused eliminator, or by evaluating both branches), but neither route forms s or constitutes untrusted dispatch (Remark 4.3).

Component-wise encoding ($C(A) + C(B)$) permits pattern matching but leaks the tag. Under component-wise encoding, the value is stored as a tagged pair: either $(L, \text{enc}_A(a, k))$ or $(R, \text{enc}_B(b, k))$, where the tag L or R is in the clear (or distinguishable by the encoding structure—for instance, A -cipher-values and B -cipher-values occupy disjoint regions of bit-string space, or carry a type identifier). The untrusted machine reads the tag and routes to the correct handler. Pattern matching is available.

But the tag is observable. Given a cipher value, the untrusted machine learns whether it encodes an A -value or a B -value. For any distribution D on $A + B$, the fraction of queries that are A -values is observable, leaking $\Pr_{v \sim D}[v \in A]$.

No intermediate encoding achieves both. Suppose an encoding achieves both tag hiding (advantage $\leq \delta'$ for some small δ') and untrusted dispatch (success probability $\geq 1 - \gamma$). Dispatch requires a selector $s : \{0, 1\}^n \rightarrow \{A, B\}$ that is correct on valid encodings with probability $\geq 1 - \gamma$. Tag hiding requires that no such function exists with advantage better than δ' . Let $p = \Pr[v \in A]$ be the prior probability of an A -value. A trivial strategy that ignores the encoding and always guesses the majority type achieves accuracy $\max(p, 1 - p)$. The advantage of s over this baseline is at least $(1 - \gamma) - \max(p, 1 - p)$. Tag hiding requires this advantage to be at most δ' . Therefore $(1 - \gamma) - \max(p, 1 - p) \leq \delta'$, which means either $\gamma \geq 1 - \max(p, 1 - p) - \delta'$ (dispatch fails often) or $\delta' \geq (1 - \gamma) - \max(p, 1 - p)$ (the tag is substantially leaked). For the balanced case $p = 1/2$, this simplifies to $1/2 - \gamma \leq \delta'$. Unbalanced distributions ($p \neq 1/2$) make the impossibility stronger: the

baseline accuracy is higher, leaving less room for the encoding to add information without leaking the tag. $\square$

Remark 4.1 (Contrast with products). Products leak correlations; sums leak the tag. The asymmetry is structural. For a product $A \times B$, the two components are always both present; the question is whether their relationship (correlation) is visible. For a sum $A + B$, only one component is present; the question is whether which one is visible. A product hides *what values co-occur*; a sum hides *what type is occupied*.

Remark 4.2 (Trusted pattern matching). Under joint encoding $C(A + B)$, pattern matching is still possible—by the trusted machine. The trusted machine decodes $\text{dec}(c)$, inspects the tag, applies the appropriate handler in plaintext, and re-encodes the result. This requires a round trip through the trusted machine for every case analysis. The cost is a communication round, not an impossibility. The impossibility is for *untrusted* pattern matching without the trapdoor.

Remark 4.3 (Oblivious elimination without dispatch). Theorem 4.2 forbids *dispatch*: selecting which of two separately held handlers to apply. It does not forbid computing the *result* of a case analysis as a cipher value with the tag hidden, which is possible in two ways, neither of which is untrusted dispatch. First, the trusted machine can compile the entire eliminator $h(\text{inl } a) = f(a)$, $h(\text{inr } b) = g(b)$ into a single cipher map $\hat{h} : C(A + B) \rightarrow C(Z)$ over the joint encoding; the untrusted machine evaluates $\hat{h}(c)$ blindly, learning neither the tag nor which branch fired. This is the efficient form of trusted pattern matching (Remark 4.2): the case analysis is compiled once at construction time instead of paid as a per-query round trip, and no trapdoor is needed at evaluation. Crucially $\hat{h}$ is a trusted construction over the whole domain $A + B$ (building it requires f , g , and the secret); the untrusted machine can evaluate it but could not have formed it, and applying one fixed map to every input is not the data-dependent selection that Theorem 4.2 rules out. Second, the untrusted machine can evaluate *both* handlers, $\hat{f}(c)$ and $\hat{g}(c)$, and obviously combine them through a cipher multiplexer (the branching construction of Example 5.3); this hides the tag as well, at the cost of evaluating both branches. In both routes the case analysis is internalized—fused into one map, or computed on both branches—so the untrusted machine never forms the selector s and never branches on the tag.

Example 4.1 (Cipher optionals). The optional type $\text{Maybe}(A) = \mathbf{1} + A$ is a sum. Under $C(\mathbf{1} + A)$, the untrusted machine cannot tell whether a cipher value encodes *Nothing* or *Just(a)*. Under $C(\mathbf{1}) + C(A)$, the untrusted machine knows whether the value is present, but not what it is. For a database application, the first encoding hides which records have null fields; the second reveals the null pattern but permits null-checking without the trapdoor.

4.4 Exponential Types

The exponential type $B^A = A \rightarrow B$ is the type of functions from A to B . The cipher exponential closes the algebra back on itself.

Proposition 4.3 (Cipher exponential is the cipher map abstraction). *For finite types A and B , the cipher exponential $C(A \rightarrow B)$ is precisely the set of cipher maps for functions $f : A \rightarrow B$. A function $\hat{f} : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a well-formed element of $C(A \rightarrow B)$ if and only if it satisfies the four cipher-map properties of Definition 3.1 (totality, representation uniformity, correctness, composability).*

Proof. A cipher map $\hat{f}$ for $f : A \rightarrow B$ is by construction a total function on $\{0, 1\}^n$ that, when restricted to valid encodings of A , produces approximate encodings of B -values (per [18]). Conversely, any total function on $\{0, 1\}^n$ that is correct (with parameter η) on some encoding of A is a

cipher map for the function it computes on the latent values. The four cipher-map properties are exactly the well-formedness conditions for membership in $\mathbf{C}(A \rightarrow B)$. $\square$

This identity has a structural consequence: every cipher type constructor in the paper, including products, sums, and exponentials, ultimately reduces to a question about cipher maps. The orbit-closure framework of Section 5 therefore applies uniformly to operations on cipher exponentials, just as it applies to operations on cipher products and sums.

The encoding-granularity knob applies to exponentials too, with its own constructor-specific cost.

Proposition 4.4 (Exponential confidentiality trade-off). *Let $f : A \rightarrow B$ and $g : C \rightarrow D$ be functions with cipher realizations $\hat{f}$ and $\hat{g}$.*

1. *Under component-wise application $(\hat{f}, \hat{g}) : \mathbf{C}(A) \times \mathbf{C}(C) \rightarrow \mathbf{C}(B) \times \mathbf{C}(D)$, the untrusted machine sees the two cipher maps applied independently and learns that both f and g are being computed (the computation pattern).*
2. *Under joint application $\widehat{(f, g)} : \mathbf{C}(A \times C) \rightarrow \mathbf{C}(B \times D)$ realized as a single cipher map for the product function $(f, g)(a, c) = (f(a), g(c))$, the untrusted machine sees one opaque cipher-map invocation and cannot determine which constituent functions were applied. This requires a dedicated cipher map of size $O(|A| \cdot |C|)$ rather than $O(|A|) + O(|C|)$.*

Proof. Part (1): The untrusted machine evaluates $\hat{f}$ and $\hat{g}$ separately and observes that two distinct cipher maps were applied. The identities of $\hat{f}$ and $\hat{g}$, as objects in $\mathbf{C}(A \rightarrow B)$ and $\mathbf{C}(C \rightarrow D)$ respectively, are themselves part of the cipher-program code visible to the untrusted machine.

Part (2): A single cipher map $\widehat{(f, g)}$ over the product domain has only one invocation, indistinguishable from any other cipher-map invocation of the same arity and shape. The product-domain construction has size proportional to $|A \times C| = |A| \cdot |C|$ rather than the sum of the component domains, per the lookup-table cost in [18, Sec. 6]. $\square$

The asymmetry in part (2) recapitulates the product-type trade-off of Proposition 4.1: hiding the computation pattern is itself an instance of joint product encoding, applied to the exponentials rather than to their inputs.

4.5 Summary: The Cipher-Type Algebra

Table 1 consolidates the per-constructor results. Three observations stand out.

Table 1: Confidentiality cost by type constructor. “Joint” encodes the constructed type as a single cipher value; “component-wise” builds the cipher type from cipher types of the components.

Constructor	Joint encoding hides	Component-wise encoding leaks
0 (void)	N/A	N/A
1 (unit)	nothing to hide	nothing to leak
$A \times B$ (Proposition 4.1)	correlations between a, b	joint distribution as $N \rightarrow \infty$
$A + B$ (Theorem 4.2)	the tag (which variant)	the tag, by impossibility
$A \rightarrow B$ (Proposition 4.4)	the computation pattern	which functions are being applied

Universal granularity knob. For each binary constructor, joint versus component-wise encoding forms a continuous spectrum (Remark 4.1, the entanglement parameter p of [18, Sec. 8.1]), and the constructor determines *what* sits on each side. The product spectrum runs from full correlation hiding to projections that leak it; the sum spectrum runs from full tag hiding to pattern matching that leaks it; the exponential spectrum runs from a single product cipher map to independent component cipher maps.

Sums are categorically different. Products and exponentials admit *intermediate* encodings: an adversary can recover correlations or computation patterns asymptotically but never exactly from a finite observation. The sum-type result is sharper: tag hiding and untrusted pattern matching cannot *coexist* at any encoding, intermediate or otherwise (Theorem 4.2). The asymmetry is structural, not quantitative: products and exponentials always have both components present so the question is which relationships are observable; sums commit to one variant per value so the question is whether the variant itself is observable.

Closure under the orbit framework. The per-constructor costs of Table 1 are local: they describe what one cipher value or one operation reveals. The next section lifts these into a uniform information-theoretic bound (Theorem 5.3) that controls what the untrusted machine learns by *actively composing* the operations exposed to it. The typed-composition discipline (Section 5.5) is the algebra’s design-time budget: the type of a cipher program bounds how deeply the untrusted machine can chase those compositions.

5 Orbit Closure and Information Leakage

The type constructors of Section 4 describe static confidentiality properties: what the untrusted machine can learn from the structure of a single cipher value. We now develop a tool for analyzing dynamic confidentiality: what the untrusted machine can learn by actively applying the cipher maps it holds.

5.1 Definition

Definition 5.1 (Orbit closure). Let $F = \{\hat{f}_1, \dots, \hat{f}_k\}$ be a set of cipher maps (total functions $\{0, 1\}^n \rightarrow \{0, 1\}^n$) available to the untrusted machine, and let $c \in \{0, 1\}^n$ be a cipher value. The *orbit closure* of c under F is the smallest set $\text{orbit}_F(c) \subseteq \{0, 1\}^n$ such that:

1. $c \in \text{orbit}_F(c)$, and
2. if $c' \in \text{orbit}_F(c)$ and $\hat{f}_i \in F$, then $\hat{f}_i(c') \in \text{orbit}_F(c)$.

Equivalently, $\text{orbit}_F(c)$ is the set of all bit strings reachable from c by composing operations in F in any order and any number of times.

Since $\{0, 1\}^n$ is finite, $\text{orbit}_F(c)$ is always finite and can be computed by iterative expansion (breadth-first search from c , applying each $\hat{f}_i$ at each step, until no new values appear).

Remark 5.1. The orbit closure depends only on the cipher maps F available to the untrusted machine and the starting cipher value c . It does not depend on the latent function, the domain, or the trapdoor. This makes it a tool for analysis by the trusted machine designer: given the operations you plan to expose, how much can the untrusted machine explore?

5.2 Monotonicity

Theorem 5.1 (Monotonicity of orbit closure). *If $F \subseteq G$ (every operation in F is also in G), then $\text{orbit}_F(c) \subseteq \text{orbit}_G(c)$ for all $c \in \{0, 1\}^n$.*

Proof. By induction on the iterative construction of $\text{orbit}_F(c)$.

Base case. $c \in \text{orbit}_F(c)$ and $c \in \text{orbit}_G(c)$ by definition.

Inductive step. Suppose $c' \in \text{orbit}_F(c)$ and $c' \in \text{orbit}_G(c)$ (inductive hypothesis). For any $\hat{f}_i \in F$, we have $\hat{f}_i(c') \in \text{orbit}_F(c)$ by Definition 5.1. Since $F \subseteq G$, $\hat{f}_i \in G$, so $\hat{f}_i(c') \in \text{orbit}_G(c)$ by Definition 5.1 applied to G .

Every element of $\text{orbit}_F(c)$ is therefore also in $\text{orbit}_G(c)$. $\square$

The interpretation is direct: exposing more operations to the untrusted machine can only enlarge (never shrink) the set of cipher values it can reach. Each additional operation is an additional edge in the reachability graph over $\{0, 1\}^n$.

Corollary 5.2. $\text{orbit}_\emptyset(c) = \{c\}$ for all c . *If the untrusted machine has no cipher maps, it learns nothing beyond the cipher value it was given.*

Proof. With $F = \emptyset$, only the base case of the orbit definition applies. $\square$

5.3 Confidentiality Bound

The orbit closure quantifies reachability. We now turn it into a confidentiality bound. The argument is a one-line counting bound on mutual information; the content is in correctly identifying *what the untrusted machine observes*.

Let X be the random latent value encoded by c , drawn from the trusted machine's input distribution. By probing c with the operations in F , the untrusted machine resolves its computation to one of the cipher values reachable from c ; write $\mathcal{V}_F(c)$ for this *observation* and model it as a random variable supported on $\text{orbit}_F(c)$. Two facts pin down its role. First, the orbit *set* is computable from c and F alone, without the trapdoor (Remark 5.1); it is therefore not itself a secret-dependent signal, and the orbit is precisely the *support* of the observation, not a set-valued random variable in its own right. Second, what varies with the latent value, and hence carries information about X , is *which* element of that support the computation resolves to, and there are at most $|\text{orbit}_F(c)|$ of those. The adversary's residual uncertainty about X is the conditional entropy $H(X \mid \mathcal{V}_F(c))$.

Theorem 5.3 (Confidentiality bound, entropy form). *Let F be a set of cipher maps available to the untrusted machine and c a cipher value. Then*

$$H(X \mid \mathcal{V}_F(c)) \geq H(X) - \log_2 |\text{orbit}_F(c)|.$$

Proof. The observation $\mathcal{V}_F(c)$ is supported on the finite set $\text{orbit}_F(c)$, so it takes at most $|\text{orbit}_F(c)|$ distinct values. The entropy of a random variable supported on a set of size M is at most $\log_2 M$ (with equality iff it is uniform), hence $H(\mathcal{V}_F(c)) \leq \log_2 |\text{orbit}_F(c)|$. Mutual information is bounded by the entropy of either argument, so

$$I(X; \mathcal{V}_F(c)) \leq H(\mathcal{V}_F(c)) \leq \log_2 |\text{orbit}_F(c)|,$$

and substituting into $H(X \mid \mathcal{V}_F(c)) = H(X) - I(X; \mathcal{V}_F(c))$ gives the claim. $\square$

Remark 5.2 (Which scale this bounds). Theorem 5.3 is an *active*-adversary bound: it limits what the untrusted machine learns by composing the operations it holds, and the controlling quantity is the orbit size, not the representation-uniformity parameter δ (Property 2). This is a different axis from the *marginal* confidentiality of a single cipher value, whose leakage is governed by δ through the entropy-ratio bound of [18, 19]. The two are complementary and do not reduce to each other: a cipher value can be marginally near-uniform ($\delta \approx 0$) yet leak its latent value through a large orbit if the exposed operation set is rich, and conversely a small orbit caps active leakage even when δ is not negligible. No value of δ bounds the orbit, and no orbit bound improves δ . The bound is loose: it ignores the functional relationships between orbit elements (the adversary knows $c' = \hat{f}(c)$, which constrains the possible latent values beyond mere set size) and the algebraic structure of F . Tighter bounds for specific operations are an open problem. Equality is approached when orbit elements behave like independent signals about X , e.g., when the cipher maps in F are modeled as random oracles [1].

Remark 5.3 (Extremes). When $|\text{orbit}_F(c)| \geq |X|$ the bound is vacuous (the subtracted term exceeds $H(X)$); in principle a rich enough operation set can identify every latent value. When $|\text{orbit}_F(c)| = 1$ (no operations available, per Corollary 5.2), the bound is tight in the strongest sense: $H(X \mid \mathcal{V}_F(c)) = H(X)$, so the untrusted machine learns nothing about X beyond what the bare cipher value already revealed.

5.4 Examples

Example 5.1 (Boolean operations). Let $X = \{0, 1\}$ (Boolean) with cipher maps $\widehat{\text{AND}}$ and $\widehat{\text{NOT}}$ available to the untrusted machine. Given a cipher value c encoding some unknown Boolean x , the untrusted machine can compute:

$$\begin{aligned} c_1 &= \widehat{\text{NOT}}(c), \\ c_2 &= \widehat{\text{AND}}(c, c_1) = \widehat{\text{AND}}(c, \widehat{\text{NOT}}(c)). \end{aligned}$$

Now, $\text{AND}(x, \text{NOT}(x)) = 0$ for all x (up to correctness η). So c_2 is an (approximate) cipher encoding of the constant FALSE. If the untrusted machine has a reference cipher value c_F known to encode FALSE (for instance, from a prior computation $\widehat{\text{AND}}(c', \widehat{\text{NOT}}(c'))$ for some other cipher value c'), it can compare c_2 with c_F .

This comparison reveals information: if $c_2 = c_F$ (same representation), the untrusted machine confirms the tautology and potentially identifies the representation index. Writing $F_0 = \{\widehat{\text{AND}}, \widehat{\text{NOT}}\}$, the orbit $\text{orbit}_{F_0}(c)$ includes c , c_1 , c_2 , and all further compositions—quickly covering a nontrivial fraction of the cipher space for small n .

Example 5.2 (Successor on a bounded type). Let $X = \{0, 1, \dots, m-1\}$ (integers mod m) with a single cipher map $\widehat{\text{succ}}$ implementing the successor function $x \mapsto (x+1) \bmod m$. Given any cipher value c encoding some x :

$$\text{orbit}_{\{\widehat{\text{succ}}\}}(c) = \{c, \widehat{\text{succ}}(c), \widehat{\text{succ}}^2(c), \dots, \widehat{\text{succ}}^{m-1}(c)\}.$$

After m applications, the orbit returns to c (since successor is cyclic on $\mathbb{Z}/m\mathbb{Z}$). The orbit has exactly m elements (assuming $\eta = 0$). The untrusted machine does not learn which element is $\text{enc}(0, k)$, but it learns the cycle structure: there exists a cycle of length m starting from c . This reveals $m = |X|$, the domain cardinality.

By Theorem 5.3, the confidentiality is $1 - m/2^n$. For $m \ll 2^n$, this is close to 1: the orbit covers a tiny fraction of the cipher space. For m close to 2^n , confidentiality degrades.

Example 5.3 (Branching). Suppose the untrusted machine holds cipher maps $\hat{f}$ and $\hat{g}$ and evaluates both on a cipher value c , obtaining $\hat{f}(c)$ and $\hat{g}(c)$. The orbit of c under $\{\hat{f}, \hat{g}\}$ includes both results plus all further compositions. If $\hat{f}$ and $\hat{g}$ are the two branches of a conditional (the “then” and “else” handlers), both cipher results are visible to the untrusted machine—even though in plaintext computing, only one branch would be taken.

This is a form of information leakage specific to trapdoor computing: the untrusted machine does not know which branch is “correct,” but it sees both outcomes, enlarging the orbit and reducing confidentiality. Mitigating this requires either (a) evaluating both branches and having the trusted machine select the correct result, or (b) encoding the branch selection as part of the cipher map (a joint encoding of the conditional). Strategy (a) doubles the computation; strategy (b) requires a dedicated cipher map construction.

Remark 5.4 (Active probing via Boolean operations). Exposing cipher Boolean operations creates an active probing risk beyond passive observation. Given cipher AND and a cipher value c , the adversary computes $\text{AND}(c, c)$, $\text{AND}(c, c')$ for other values c' , and checks structural consistency. Over many probes, the adversary partitions cipher values into equivalence classes corresponding to latent Boolean values. Each probe enlarges the orbit, and confidentiality degrades per Theorem 5.3. This is a concrete instance of orbit closure: Boolean operations are probing tools that trade functionality for confidentiality.

5.5 Typed Composition Chains

The active probing risk motivates controlling the orbit by construction. The orbit grows because the adversary can feed cipher map outputs back as inputs, composing operations to reach new cipher values. A direct mitigation: *prevent self-composition by typing the cipher spaces*.

Definition 5.2 (Typed composition chain). A *typed composition chain* of depth k is a sequence of cipher maps $\hat{f}_0, \dots, \hat{f}_{k-1}$ where $\hat{f}_i : \mathcal{C}(A)_i^{a_i} \rightarrow \mathcal{C}(A)_{i+1}$ has arity $a_i \geq 1$ over the cipher space $\mathcal{C}(A)_i$, and the cipher spaces $\mathcal{C}(A)_0, \dots, \mathcal{C}(A)_k$ are *distinct* (disjoint hash value sets, different secrets).

Since $\mathcal{C}(A)_i \neq \mathcal{C}(A)_j$ for $i \neq j$, the output of $\hat{f}_i$ cannot be fed into $\hat{f}_j$ for $j \neq i + 1$. The adversary can compose at most k operations in sequence and cannot loop.

Proposition 5.4 (Orbit bound for typed chains). Let $V_0 \subseteq \mathcal{C}(A)_0$ be a set of m initial cipher values available to the adversary, and let $F = \{\hat{f}_0, \dots, \hat{f}_{k-1}\}$ be a typed composition chain of depth k with arities $a_0, \dots, a_{k-1}$. Define $N_0 = m$ and $N_{i+1} = N_i^{a_i}$. Then

$$|\text{orbit}_F(V_0)| \leq \sum_{i=0}^k N_i,$$

a finite bound determined entirely by $(k, m, a_0, \dots, a_{k-1})$ and independent of the cipher-space sizes $|\mathcal{C}(A)_i|$. Two useful special cases:

- **Single-value start** ($m = 1$), any arities: $|\text{orbit}_F(V_0)| \leq 1 + k$.
- **Unary chain** ($a_i = 1$), any m : $|\text{orbit}_F(V_0)| \leq m(k + 1)$.

Substituting $|\text{orbit}_F(V_0)| \leq \sum_i N_i$ into Theorem 5.3 bounds the corresponding confidentiality loss at $\log_2 \sum_i N_i$ bits.

Proof. Partition $\text{orbit}_F(V_0)$ by level: $L_0 = V_0 \subseteq \mathcal{C}(A)_0$, and $L_{i+1} \subseteq \mathcal{C}(A)_{i+1}$ is the image of $\hat{f}_i$ applied to all a_i -tuples of elements of L_i . Distinct cipher spaces prevent back-references, so every orbit element lies in exactly one L_i . Since $|L_i| \leq |L_{i-1}|^{a_i-1}$, induction gives $|L_i| \leq N_i$, and therefore $|\text{orbit}_F(V_0)| = \sum_i |L_i| \leq \sum_i N_i$. For $m = 1$, $N_i = 1$ at every level (identical inputs produce one output for a deterministic cipher map), so the sum is $1 + k$. For $a_i = 1$, $N_i = m$ at every level, so the sum is $m(k + 1)$. $\square$

Example 5.4 (Depth-limited Boolean search). Consider Boolean search with maximum query depth 2 (e.g., $w_1 \wedge w_2$ but not $w_1 \wedge w_2 \wedge w_3$):

- Cipher sets output to $\mathcal{C}(\text{Bool})_0$.
- $\text{AND}_0 : \mathcal{C}(\text{Bool})_0 \times \mathcal{C}(\text{Bool})_0 \rightarrow \mathcal{C}(\text{Bool})_1$ (binary, $a_0 = 2$).
- $\text{AND}_1 : \mathcal{C}(\text{Bool})_1 \times \mathcal{C}(\text{Bool})_1 \rightarrow \mathcal{C}(\text{Bool})_2$ (binary, $a_1 = 2$).
- No AND_2 is defined.

Starting from a single cipher Boolean c (one membership-query output), the orbit has size at most $1 + k = 3$: namely $\{c, \text{AND}_0(c, c), \text{AND}_1(\text{AND}_0(c, c), \text{AND}_0(c, c))\}$. With m membership-query outputs as initial values, the bound is $m + m^2 + m^4$, still finite and controlled by the chain depth. Contrast with a single shared cipher Boolean space, where the adversary can self-compose indefinitely and the orbit grows up to 2^n .

The general principle: **the type system controls the orbit**. By choosing how many cipher space levels to define, the system designer sets the maximum composition depth and therefore the maximum orbit size and minimum confidentiality. This is a compile-time (construction-time) decision with no runtime cost: the untrusted machine simply does not have cipher maps for deeper levels.

6 Realizing Cipher Programs

The type-theoretic results of Sections 4 and 5 bound what any cipher-program realization can achieve. We realize cipher programs as expression trees: a program is a DAG whose selected nodes are replaced by cipher maps and whose remaining operations run as plain code on the cipher bytes. This subsumes iterated constructions (a self-loop is one cut point applied repeatedly) and reduces to a Turing-machine-style realization in the degenerate case where the only cipher map is the transition step. We discuss only the expression-tree case here.

6.1 Expression-Tree Decomposition

The type-theoretic trade-offs of Sections 4.2 and 4.3 have a direct operational realization: given a program represented as an expression tree, which subtrees should be replaced by cipher maps?

Definition 6.1 (Cipher node annotation). Given a program P represented as an expression tree, a *cipher node annotation* marks a subset of nodes $\{n_1, \dots, n_k\}$ in the tree. Each marked node n_i , together with its entire subtree, is replaced by a cipher map: the subtree is evaluated at construction time (on the trusted machine) over a specified domain, and the resulting mapping is stored as a cipher map backed by a perfect hash function [6]. Unmarked nodes above the cipher nodes become a *combiner*: itself a cipher map built over the hash-value outputs of the components.

The placement of cipher node annotations controls the encoding granularity:

- **Annotation at the root:** the entire program is one cipher map. Maximum confidentiality (no intermediate values exposed), maximum space ($O(|X|)$ where X is the input domain of the full program).
- **Annotation at the leaves:** each leaf function is a separate cipher map. The combiner chains them. Intermediate cipher values are visible to the untrusted machine, leaking correlations between components (as in the product-type trade-off of Section 4.2).
- **Annotation at intermediate nodes:** a spectrum between the two extremes. Deeper annotations absorb more computation into cipher maps (more confidential, more space); shallower annotations expose more structure (less space, more leakage).

Remark 6.1 (CIPHERING PROPAGATES UPWARD THROUGH THE TREE). Cipher nodes cannot be chosen freely leaf-by-leaf. Replacing a subtree with a cipher map makes its output a cipher value, and any ancestor that *semantically consumes* that value must then be a cipher map too, since the untrusted machine cannot decode it. Moreover a cipher map takes cipher values as inputs, so once a node is ciphered its *sibling* inputs must be cipher values as well. The two forces together make the ciphered region upward-closed along data-flow edges with the cut points as its lower frontier (the combiner of Definition 6.1 is exactly this region); only pure plumbing—moving, pairing, or returning a cipher value untouched—may remain plaintext. Because every value in a single-rooted expression tree meets at the root, a single cut whose result reaches the output through semantic operations ciphers the entire tree (Figure 1). Control flow is the sharp case. In `if e then r1 else r2` with e ciphered, the test is a cipher Boolean, and branching on it on the untrusted machine is untrusted dispatch on the sum $1 + 1$, forbidden by Theorem 4.2: it would leak the test. The conditional must instead become a cipher multiplexer that obviously selects between r_1 and r_2 (Remark 4.3), which forces *both* branches to be cipher values and both to be evaluated—exactly why the tracing pass below computes every branch rather than the one the plaintext program would have taken.

Remark 6.2 (SHARED VARIABLES AND COMPOSITIONAL LEAKAGE). When two cipher nodes f and g share an input variable x , the untrusted machine observes two cipher values $\text{enc}_f(x, k_f)$ and $\text{enc}_g(x, k_g)$ derived from the same latent x . Even if both encodings are individually δ -close to uniform, the joint distribution of $(\text{enc}_f(x), \text{enc}_g(x))$ preserves the correlation (as in [18, Prop. 8.1]). The only mitigation is to merge f and g into a single cipher node (eliminating the shared intermediate) or to inject noise queries that dilute the correlation signal.

Implementation via tracing. A practical realization uses Python function decorators and run-time tracing. Functions marked with a `@cipher_node` decorator are intercepted during a tracing pass over the input domain. The tracing records which cipher nodes are called, with what arguments, and what they return. From this trace, the system:

1. builds a cipher map for each traced cipher node,
2. builds a combiner cipher map over the hash-output tuples of the components,
3. infers the input routing (which program arguments feed which cipher node) from the traced call patterns.

Short-circuit evaluation (e.g., Python’s `x if cond else y`) is handled by computing all component cipher maps for every input, even when the original program skips some branches. The combiner’s domain includes hash tuples for both branches. The resulting composed cipher program evaluates entirely on the untrusted machine with no round trips to the trusted machine.

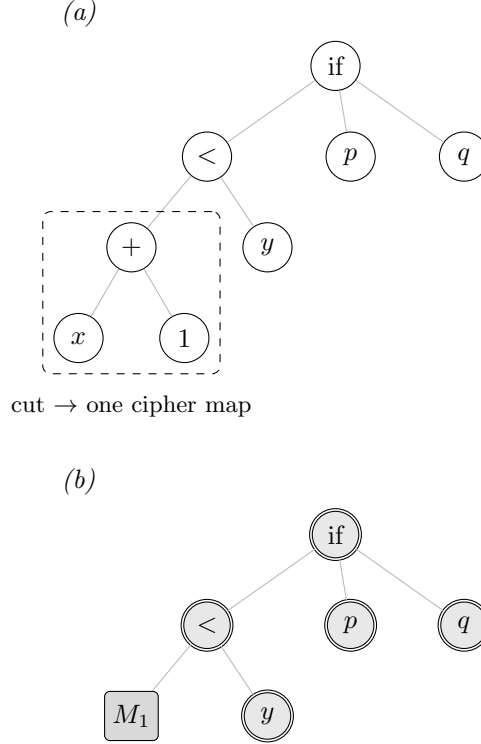


Figure 1: Ciphering propagates up an expression tree, here for `if (x + 1 < y) then p else q`. (a) The plaintext tree; cutting at `+` rolls the subtree `x + 1` into a single cipher map. (b) The cut subtree is now one opaque cipher map M_1 (rounded box); its cipher output forces `<` to become a cipher comparison, which pulls in the sibling `y`; the cipher Boolean then forces `if` into a cipher multiplexer, which pulls in both branches `p` and `q`. Double-circled shaded nodes are cipher nodes; the entire tree above the cut is ciphered.

6.2 Branching and the Limits of Oblivious Control

Control flow is where the realization meets its limits, and the boundary is exactly the sum-type impossibility of Section 4.3. A conditional `if e then P else Q` over a cipher Boolean e becomes a cipher multiplexer $\text{ite} : \mathbf{C}(\mathbf{2}) \times \mathbf{C}(R) \times \mathbf{C}(R) \rightarrow \mathbf{C}(R)$, $\text{ite}(t, a, b) = a$ if t else b , where $\mathbf{2} = \mathbf{1} + \mathbf{1}$ is the Boolean type. The untrusted machine evaluates ite blindly and never learns t ; the cost is hidden in its *inputs*, since materializing both $\mathbf{C}(R)$ arguments means evaluating both branches.

Proposition 6.1 (No oblivious short-circuit). *Evaluate `if e then P else Q` on the untrusted machine over a cipher Boolean $e = \text{enc}(t, k)$, producing a cipher value of the result type. If the untrusted machine evaluates only the branch selected by t , then it has performed untrusted pattern matching on $\mathbf{1} + \mathbf{1}$ and, by Theorem 4.2, leaks t with advantage at least $1 - \delta$. Hiding t therefore forbids short-circuiting: the untaken branch’s contribution cannot be skipped. It is paid instead either at evaluation time—evaluate both branches and combine them through ite —or at construction time, by a fused eliminator over the joint domain (Remark 4.3).*

Proof. Evaluating only the selected branch makes the untrusted machine’s action (run P versus run Q) a function $s(\text{enc}(v, k))$ that equals t on valid encodings; this is the selector of Theorem 4.2, a tag distinguisher, so t leaks. Hiding t requires the untrusted machine’s run-time behavior to be independent of t , so it cannot run one branch in place of the other. Two t -independent behaviors still

compute the result: evaluate both branches and apply `ite` (which selects via a cipher map without exposing t), or evaluate a single fused cipher map for the whole conditional whose construction already ranged over both branches. In either case the untaken branch is evaluated—at run time or at construction time—never skipped. $\square$

The plaintext efficiency of `if`—run only the taken branch—is therefore precisely what cannot be done obliviously. Three regimes follow, by how the branches scale.

Bounded, fixed-shape branches. When P and Q are straight-line (or themselves bounded conditionals), the untrusted machine performs a *fixed*, data-independent set of operations: it evaluates every branch and muxes. Confidentiality is free; the cost is that k nested conditionals evaluate all 2^k leaves. Figure 1 is the single-conditional case.

Total-but-partial branches. Evaluating the untaken branch is safe even when it would be undefined in plaintext—`if  $x \neq 0$  then  $y/x$  else 0` evaluates y/x at $x = 0$ —because cipher maps are total: the spurious evaluation returns a noise value (Proposition 7.1) that the multiplexer discards. Totality (Property 1) is what makes both-branches evaluation correct.

Unbounded, data-dependent control. When the number of operations depends on a cipher value—a loop `while  $c$  do ...` or a recursion whose depth is data dependent—the operation set can no longer be fixed. Two options remain: unroll to a *public* bound N and obviously mask the iterations past the true count (cost $O(N)$ regardless of the data, tag hidden), or test the cipher condition each step to decide whether to continue, which is untrusted dispatch per step and leaks the trajectory (the trip count, a function of the data). There is no oblivious unrolling without a public bound.

These regimes are one statement about *shape*: the expression-tree realization is oblivious exactly because it compiles to a fixed, data-independent computation graph. Branching is the atomic data-dependent decision, handled obliviously only by evaluating both sides; unbounded data-dependent control cannot be flattened to a fixed graph, and there the trajectory must leak. That unbounded case is a cipher Turing machine, whose per-step (state, direction) leakage is the loop leak above; it is why this paper takes the fixed-graph expression tree, not the Turing machine, as the realization (cf. Section 6.3).

Remark 6.3 (The multiplexer is not free). As a cipher map, `ite` : $\mathbf{C}(\mathbf{2}) \times \mathbf{C}(R) \times \mathbf{C}(R) \rightarrow \mathbf{C}(R)$ has domain $2|R|^2$, so oblivious selection over a result type R costs $O(|R|^2)$ space—the product-encoding cost of Proposition 4.1 resurfacing inside control flow. Fusing the whole conditional into one cipher map over the input domain X costs $O(|X|)$ instead; which is cheaper is the granularity choice of Section 6.1. And if P and Q return different variants, `ite`’s output is a cipher sum, and the dispatch obligation of Theorem 4.2 reappears at the next consumer rather than dissolving.

6.3 Cut-Point Structure

Definition 6.2 (Cut point). Given a plaintext computation P represented as a directed acyclic graph with functions at nodes and data flow along edges, a *cut point* is a node whose subgraph of ancestors is replaced, during construction, by a single cipher map. Edges entering the cut point carry plaintext on the trusted machine and cipher values on the untrusted machine; edges leaving the cut point carry cipher values throughout. A *cut-point set* is a collection of cut points such that every input of P reaches at least one of them; the cipher maps at the cut points together with the unmarked nodes above them (which evaluate as plain code on the cipher bytes) compute P on the untrusted machine.

What leaks above the cut points. Exactly what remains plaintext above the cut points is the leakage profile: the collection of intermediate cipher values flowing between cipher nodes, plus whatever the unmarked plain operations reveal about those values (equality checks, branching, dictionary lookups). Multiple cut points sharing a latent input leak that input’s correlation structure across their outputs, per the product-encoding trade-off (Proposition 4.1).

What bounds the orbit. A typed-chain decomposition (distinct cipher spaces between consecutive cut points) inherits the bound $\sum_i N_i$ of Proposition 5.4; for a depth- k tree with binary combinators starting from m distinct latent inputs, the bound is $\sum_{i=0}^k m^{2^i}$. Decompositions that reuse a single cipher space across cut points (equivalently, that admit a self-loop in the cipher-map dependency graph) fall outside the typed-chain hypothesis and require a separate argument; for deterministic iterated computations of length T , a single-trajectory bound of $T + 1$ orbit elements per starting cipher value is straightforward but the leakage profile is severe: every per-step plaintext signal needed to advance the loop (selectors, indices, halting time) is observable. This is the unbounded-control regime of Section 6.2: a self-loop is exactly the data-dependent control that cannot be flattened to a fixed graph.

Example 6.1 (Regex matching as a typed cut-point chain). Consider matching a fixed regular expression R of size r against a string $x \in \Sigma^\ell$, with DFA state set Q_R (in the worst case $|Q_R| = 2^{O(r)}$ via subset construction; for many regexes much smaller). The natural typed-chain decomposition places ℓ cut points $\hat{\tau}_0, \dots, \hat{\tau}_{\ell-1}$ with $\hat{\tau}_i : \mathbf{C}(Q_R)_i \times \mathbf{C}(\Sigma) \rightarrow \mathbf{C}(Q_R)_{i+1}$, each of space $O(|Q_R| \cdot |\Sigma|)$, total $O(\ell \cdot |Q_R| \cdot |\Sigma|)$. The orbit is bounded by $1 + \ell$ from Proposition 5.4 (single state value at each level), and the leakage above the cut points is the $(\ell + 1)$ -tuple of intermediate cipher state values $c_0, \dots, c_\ell$. The same computation can be expressed as a self-loop on a single cipher space $\mathbf{C}(Q_R \times \Sigma)$ with a single transition cipher map of space $O(|Q_R|^2 \cdot |\Sigma|)$ (constant in ℓ), but the leakage then includes a plaintext (next-state-selector, direction) pair per step, redacting only the tape contents. The choice between the typed chain and the self-loop is therefore not about space (the typed chain is asymptotically smaller per cut point once $\ell > |Q_R|$) but about *which structure leaks*.

7 Cipher Boolean Algebra and Evaluation

The cipher Boolean type $\mathbf{C}(\text{Bool})$ specializes the sum-type trade-off to the most fundamental algebraic structure and provides the concrete instantiation we evaluate on a real corpus.

7.1 Cipher Boolean Type

Definition 7.1 (Cipher Boolean type). A cipher Boolean type over $\{0, 1\}^n$ is a partition of the hash space into three disjoint regions:

- $T \subset \{0, 1\}^n$: the True region ($|T|/2^n = p_{\text{true}}$),
- $F \subset \{0, 1\}^n$: the False region ($|F|/2^n = p_{\text{false}}$),
- $N = \{0, 1\}^n \setminus (T \cup F)$: the noise region.

The partition is derived from a secret via a pseudorandom permutation. Without the secret, membership in T , F , or N is indistinguishable from random.

Remark 7.1 (Generalization beyond Bool). Definition 7.1 generalizes to any finite type Y . A cipher type $\mathbf{C}(Y)$ partitions $\{0, 1\}^n$ into $|Y|$ value regions plus a noise region, with $|R_y|/2^n \propto \Pr[Y = y]$

(Shannon-optimal allocation per the source coding theorem [14]). Operations $f : Y_1 \times \dots \times Y_k \rightarrow Z$ become cipher maps over the product of input regions. The cipher Boolean is the special case $Y = \{\text{True}, \text{False}\}$; the same construction applies to enumerated types, bounded integers, or any codomain where the output distribution is known.

7.2 Noise Unreliability

The noise region N plays a structural role beyond mere error tolerance.

Proposition 7.1 (Noise unreliability). *Let AND, OR, NOT be cipher maps implementing Boolean operations over the cipher Boolean type. If any input is a noise value $n \in N$, the output is uncorrelated with the intended Boolean result: it lands in T , F , or N according to the allocation ratios p_T , p_F , p_N , independent of the other input’s value.*

Proof. The AND, OR, and NOT cipher maps are constructed over the domain $T \cup F$. An input from N is outside this domain. Since the cipher map is total (backed by a PHF with random noise in unused slots), a noise input maps to a random slot whose contents decode to a value in T with probability p_T , in F with probability p_F , and in N with probability p_N . For the standard allocation ($p_T = 0.05$, $p_F = 0.90$, $p_N = 0.05$), a noise input produces False 90% of the time, True 5%, and noise 5%. Crucially, this output is independent of the other (valid) input: the cipher map cannot “correct” for a noise input because it does not know which inputs are noise. $\square$

Remark 7.2 (Noise prevents output certainty). The noise region ensures that an adversary observing a sequence of Boolean cipher map evaluations cannot achieve certainty about any individual result. Consider an adversary that repeatedly ANDs a cipher value c with random cipher values $r_1, r_2, \dots$. If $c \in T$ (True), then $\text{AND}(c, r_i)$ produces True when $r_i \in T$ and False when $r_i \in F$. But when $r_i \in N$ (5% probability), the output is noise, which the adversary cannot distinguish from a legitimate False. After k ANDs, the adversary has seen some True and some False/noise results, but cannot determine with certainty whether any given False result is real or noise-induced.

Dually, an adversary that repeatedly ORs c with random values sees True for $c \in T$ (always True), but again noise inputs produce unpredictable outputs. For $c \in F$, the OR produces True only when $r_i \in T$ (5%), False when $r_i \in F$ (90%), and noise when $r_i \in N$ (5%). The noise fraction creates a floor of uncertainty in any statistical test the adversary performs.

Practical allocation. The cipher Boolean allocation $|T|/2^n = 0.05$, $|F|/2^n = 0.90$, $|N|/2^n = 0.05$ is motivated by the set membership use case. A cipher set $\hat{1}_A : \mathcal{C}(X) \rightarrow \mathcal{C}(\text{Bool})$ maps domain elements to cipher Booleans. Members are forced into T by the construction. Non-members land randomly: 90% decode as False (correct rejection), 5% as True (false positive), 5% as noise. The false positive rate equals p_{true} and is a system parameter, not dependent on the set size or universe.

Composition with set operations. Boolean search queries like “ $w_1 \wedge w_2 \wedge \neg w_3$ ” are evaluated entirely on the untrusted machine by chaining cipher set evaluations through cipher AND/OR/NOT maps. Each Boolean operation is itself a cipher map (constructed via PHF over the $|T \cup F|^2$ input pairs for AND/OR, or $|T \cup F|$ for NOT). The untrusted machine sees opaque hash values flowing through opaque total functions at every stage.

7.3 Experimental Validation

A reference implementation (cipher-maps, Python with PHF backend via phobic) validates the construction on the 20 Newsgroups corpus [11] (18,266 documents, 58,903 unique words). With

$n = 8$ bits, $|T| = 13$, $|F| = 230$, the AND and OR cipher maps each have $(13 + 230)^2 = 59,049$ entries. All numbers below are means $\pm$ one standard deviation across five independent seeds; hardware: x86_64, single-threaded, Python 3.12.

Boolean search at scale. Table 2 shows precision and recall for representative Boolean queries over a fixed random 5,000-document sample, with all Boolean operations evaluated on the untrusted machine. Build rate: 1835 documents per second (mean over seeds). Plaintext baseline columns are 1.00/1.00 by construction (the cipher Boolean type is a sound over-approximation of set membership); they are included to make the cipher-side recall degradation explicit.

Table 2: Boolean encrypted search, 5,000 documents, $p_T = 0.05$, $n_{\text{seeds}} = 5$. Plaintext baseline is 1.00/1.00 by construction.

Query	Precision	Recall	FP count	Time
Single term	0.408 ± 0.014	1.000 ± 0.000	233 ± 13	0.04s
2-term AND	0.262 ± 0.024	1.000 ± 0.000	43 ± 5	0.13s
3-term AND	0.291 ± 0.043	1.000 ± 0.000	15 ± 3	0.22s
OR	0.365 ± 0.010	0.959 ± 0.010	437 ± 16	0.13s
OR AND NOT	0.330 ± 0.008	0.909 ± 0.010	402 ± 15	0.26s

AND queries maintain perfect recall while false positives decrease ($233 \rightarrow 15$ from single term to 3-term AND). OR and NOT queries lose recall (0.96 and 0.91) due to noise propagation: when a cipher set returns a noise value $n \in N$, the subsequent OR or NOT operation produces noise or an unpredictable result, effectively dropping the document from the result set. Precision values reflect the $p_T = 0.05$ false-positive rate per cipher set; for a true set of size k in a corpus of size N , the expected precision is approximately $k/(k + p_T(N - k))$, which for our representative single-term query yields ≈ 0.4 .

FPR compounding vs. Bernoulli model. The Bernoulli composition model [17] predicts that k independent AND operations yield $\text{FPR}_k = p_T^k$ and k independent OR operations yield $1 - (1 - p_T)^k$. Table 3 reports empirical FPRs over 1,000 trials per seed across five seeds. OR chains match theory tightly across all k (ratio in $[0.90, 1.02]$). AND chains diverge sharply with depth: the empirical FPR exceeds p_T^k by a factor of three at $k = 2$, by $35\times$ at $k = 3$, and by more than four orders of magnitude at $k = 5$.

The asymmetry has a structural explanation. A Bernoulli AND test of k independent draws is itself k -independent, but cipher Boolean AND is a *deterministic cipher map*: once a chain produces an intermediate cipher Boolean value c' , feeding c' into the next AND produces a fixed (not redrawn) output. Independence holds within a single cipher set (each non-member draws a fresh hash) but breaks when the same cipher Boolean cipher map is composed with itself k times. The OR chain sees the same composition pattern but stays close to theory because the Bernoulli OR formula is dominated by the larger $1 - (1 - p_T)^k$ term, which is comparatively insensitive to within-chain dependence; AND collapses to p_T^k , a small number that is easy to overshoot. A formal correction term is open work; the empirical floor of $\sim 10^{-3}$ at $k \geq 3$ is one of this paper’s concrete contributions to the cipher-Boolean error model.

Encoding granularity. We built a 7-function loan approval pipeline at three granularity levels (Table 4). The root annotation uses a single PHF-backed cipher map over the full domain; intermediate and leaf annotations use seed-search-backed component cipher maps composed via tracing.

Table 3: FPR through Boolean chains, 200-document subset, 1,000 trials per seed, $n_{\text{seeds}} = 5$. Last column is empirical / theoretical ratio.

Chain	k	Theoretical FPR	Empirical FPR	Ratio
AND	1	5.00×10^{-2}	0.0504 ± 0.0026	1.01
AND	2	2.50×10^{-3}	0.0074 ± 0.0029	2.96
AND	3	1.25×10^{-4}	0.0044 ± 0.0021	35.2
AND	4	6.25×10^{-6}	0.0016 ± 0.0018	256
AND	5	3.13×10^{-7}	0.0038 ± 0.0029	12,160
OR	1	0.0500	0.0448 ± 0.0086	0.90
OR	2	0.0975	0.0912 ± 0.0078	0.94
OR	3	0.1426	0.1452 ± 0.0051	1.02
OR	4	0.1855	0.1724 ± 0.0101	0.93
OR	5	0.2262	0.2098 ± 0.0062	0.93

Seed-search occasionally fails to find a valid hash assignment for a given component domain, which is why the intermediate row reports $n = 2$ successful seeds rather than 5.

Table 4: Encoding granularity over the 150-input loan-approval domain, $n_{\text{seeds}} = 5$ attempted (successful seed count shown for the seed-search levels).

Level	Build (s)	Space	Bits/elem	Intermediates
Root (1 cipher map)	0.003 ± 0.000	694 B	37.0	0
Intermediate (3 groups, $n = 2$)	1.67 ± 1.16	138 B	7.4	3
Leaf (7 nodes)	1.14 ± 1.45	140 B	7.5	7

As expected from the PHF construction, all three produce zero errors on the full domain (150 inputs); each cipher map is built over its declared input set, so on-domain inputs are exact lookups by construction. The root annotation is roughly $400\times$ faster (PHF lookup vs. per-element seed search) and exposes zero intermediate cipher values. Counter-intuitively, the leaf and intermediate annotations use *less* total space (≈ 140 B vs. the root’s 694 B): each component cipher map’s domain is smaller than the product domain of the full program, and the seed-search backend stores one n -bit seed per element. The space-leakage trade-off is therefore not a uniform “coarser is larger”; coarser annotation buys hidden intermediate values at the cost of construction time and a build that requires the full input domain up front, while leaf annotations build smaller cipher maps that compose dynamically but expose every intermediate hash flow.

8 Discussion

Relationship to cipher maps. This paper extends the cipher map framework of [18] in two directions: inward (what happens when cipher values carry algebraic structure) and outward (what happens when the untrusted machine actively explores the cipher space). The cipher map paper focuses on individual maps and their four properties; this paper studies how those properties interact with type constructors and how confidentiality degrades under active use. The two frameworks measure confidentiality on complementary axes. The cipher map paper bounds the leakage of a single cipher value’s *marginal* distribution through the representation-uniformity parameter δ [18, Prop. 5.1], and analyzes cross-instance *collision* leakage when many cipher values are observed together [18, §8.2]; the entropy-ratio treatment is developed further in [19]. Our orbit-closure bound

(Theorem 5.3) measures a third, orthogonal axis: the *reachability* leakage an adversary accrues by actively composing the operations it holds. The three are complementary, not competing: a cipher value can be marginally uniform ($\delta \approx 0$), collision-free, and still leak through a large orbit if the exposed operation set is rich.

Relationship to Bernoulli data types. The accuracy-side analysis of algebraic types over approximate values is developed in [17], which studies the Kronecker factorization of joint confusion matrices for product types and the error propagation through sum-type constructors. That paper asks: how does η change when types are composed? This paper asks the complementary question: how does confidentiality change when types are composed? Together, they characterize the full trade-off space (accuracy $\times$ confidentiality) for algebraic types over trapdoor computing.

The encoding granularity principle. The product and sum trade-offs in Sections 4.2 and 4.3 are instances of a single principle developed in [18, Sec. 8.1]: coarser encoding granularity (encoding more values as a single unit) yields better confidentiality but fewer operations available to the untrusted machine. The entanglement parameter p [18, Sec. 8.1] controls the spectrum: $p = 1$ (component-wise) gives maximum functionality and minimum confidentiality; $p = k$ (whole-state) gives maximum confidentiality and minimum functionality.

Open questions.

1. *Recursive types.* Lists, trees, and other recursive types are built from sums and products. The sum-type impossibility (Theorem 4.2) implies that recursive traversal (which requires pattern matching at each node) either leaks the structure (which constructors are used at each level) or requires a trusted-machine round trip at each node. Quantifying the total leakage for a recursive traversal of depth d is open.
2. *Tight orbit bounds for specific constructions.* Theorem 5.3 gives a general upper bound on confidentiality in terms of orbit size. For specific cipher map constructions (e.g., the trapdoor boolean algebra of [18]), tighter bounds may be possible by exploiting the structure of the construction.
3. *Plain operations on cipher bytes.* The expression-tree realization conflates two roles for the unmarked nodes above the cut points: a cipher-map combiner (built at construction time over the product of cipher hash spaces) and plain code that runs on the cipher bytes at evaluation time. Characterizing what each plain operation leaks (equality checks reveal collision structure; dictionary lookups reveal hash bucket membership) and when the trade with construction-time cost is preferable to a precomputed combiner cipher map is open.
4. *Partial sum-type hiding.* Theorem 4.2 shows that full tag hiding and full untrusted pattern matching are mutually exclusive. Is there a meaningful intermediate notion—hiding the tag from some adversary classes while permitting restricted pattern matching? For instance, one could imagine a construction where the tag is hidden against passive adversaries (who only observe cipher values) but available to active adversaries (who evaluate cipher maps).
5. *Cut-point placement as an optimization.* Section 6.3 describes cut points as a design knob over the computation DAG. Formulating cut-point selection as a constrained optimization, subject to the orbit bound of Proposition 5.4 and construction-time space budgets, would turn the choice into a solvable problem rather than a designer’s judgement call.

9 Conclusion

Cipher types form an algebra with a coherent confidentiality calculus. We surveyed the algebra constructor by constructor: products leak correlations, sums force a categorical impossibility (tag hiding and untrusted pattern matching cannot coexist), and the cipher exponential is the cipher map abstraction itself, closing the algebra back on the trapdoor framework. Across the algebra, a uniform information-theoretic bound holds: the conditional entropy of the latent value given the untrusted machine’s view is at least $H(X) - \log_2 |\text{orbit}|$. A typed-composition discipline turns this bound into a design-time budget; these results apply to any cipher map construction, not just specific implementations.

The algebra realizes at the program level as expression-tree decomposition. A cipher program is a composition of cipher maps and plain code over opaque bit strings: cipher-node annotations on a Python program select the cut points where cipher maps replace plaintext subtrees, while unmarked nodes run as plain code on the cipher bytes. The placement of annotations selects which intermediate structure the untrusted machine observes; the type discipline of the cut-point cipher spaces bounds the orbit per Proposition 5.4.

On the practical side, cipher Boolean types with AND/OR/NOT as cipher maps enable Boolean search queries evaluated entirely on the untrusted machine. Experiments on the 20 Newsgroups corpus (5,000 documents, five seeds) show perfect recall for AND queries and noise-driven recall loss to ≈ 0.91 for queries containing OR or NOT. The most striking empirical finding is that AND-chain FPR diverges from the Bernoulli independence prediction by orders of magnitude: empirical FPR exceeds p_T^k by $35\times$ at $k = 3$ and by over $10^4\times$ at $k = 5$, while OR chains stay within 10% of theory at every depth. This points to a structural correction term for composed deterministic cipher maps as future work. The encoding granularity spectrum is measurable: root annotation (one cipher map for the entire program) is roughly $400\times$ faster than leaf annotation (one cipher map per function), exposes no intermediates, but consumes more space than the smaller component maps; the trade-off is not a single axis but a multi-dimensional choice over (build time, total space, intermediate exposure).

Acknowledgments

The algebraic cipher types concept originates in a 2019–2022 C++ notebook by the author, which explored cipher Booleans, cipher unions, and several iterated constructions through concrete implementations. The present paper extracts and formalizes the theoretical content from that notebook, building on the cipher map framework developed in [18].

References

- [1] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.
- [2] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [3] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *Theory of Cryptography Conference*, pages 253–273, 2011.

- [4] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security*, pages 668–679, 2015.
- [5] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, pages 79–88, 2006.
- [6] Michael L Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.
- [7] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.
- [8] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.
- [9] Paul Grubbs, Thomas Ristenpart, and Vitaly Shmatikov. Why your encrypted database is not secure. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, pages 162–168, 2017.
- [10] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: Ramification, attack and mitigation. In *Proceedings of the 19th Network and Distributed System Security Symposium*, 2012.
- [11] Ken Lang. NewsWeeder: Learning to filter netnews. In *Proceedings of the 12th International Conference on Machine Learning*, pages 331–339, 1995.
- [12] Muhammad Naveed, Seny Kamara, and Charles V Wright. Inference attacks on property-preserving encrypted databases. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security*, pages 644–655, 2015.
- [13] Andrei Sabelfeld and Andrew C Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.
- [14] Claude E Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [15] Geoffrey Smith. On the foundations of quantitative information flow. In *Proceedings of the 12th International Conference on Foundations of Software Science and Computational Structures*, pages 288–302, 2009.
- [16] Dawn Xiaodong Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, pages 44–55, 2000.
- [17] Alexander Towell. Bernoulli data types: Approximate algebraic types with quantitative error, 2026. Manuscript in preparation.
- [18] Alexander Towell. Cipher maps: Total functions as trapdoor approximations, 2026. Manuscript in preparation; cross-references (Def. 3.1, §4–§5, §6, §8.1, Thm. 4.2) follow the 2026-05 revision and may shift in later versions.

- [19] Alexander Towell. Maximizing confidentiality under trapdoor computing: Encoding granularity and optimal space, 2026. Manuscript in preparation.
- [20] Alan M Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936.
- [21] Andrew C Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science*, pages 160–164, 1982.